

01 · REST: primer contacto con la API

Scrapear una **API REST** es casi siempre mejor que parsear HTML: los datos llegan ya estructurados (JSON), no se rompen cuando cambia la maquetación de la web y se transfiere mucho menos volumen.

Toda la sesión trabaja contra la API de **tienda-virtual** (`http://localhost:3000/api`). La app debe estar corriendo:

```
cd ../tienda-virtual && npm run dev
```

Un solo endpoint, `GET /api/health`, responde **sin credenciales**; todos los demás `/api/*` piden el header `x-api-key`. En este notebook lo usamos sin más como "el header que pide esta API"; su gestión a fondo (claves inválidas, scopes, revocación) es el notebook 03.

Progresión de la sesión:

Notebook	Qué se agrega
Este	Primer contacto: <code>health</code> , un recurso y una colección
<code>02_rest_paginacion</code>	Recorrer una colección completa con paginación
<code>03_rest_autenticado_api_key</code>	La API key a fondo: 401, claves inválidas, scopes, revocación
<code>04_rest_autenticado_oauth</code>	OAuth2 (<code>client_credentials</code>): token Bearer que expira
<code>05_graphql_consultas_basicas</code>	Lo mismo con GraphQL : primera query
<code>06_graphql_consultas_anidadadas</code>	GraphQL con variables, campos anidados y OAuth
<code>07_playwright_paginas_con_login</code>	Sin API: Playwright con login para páginas protegidas

```
import csv
import os

import requests

BASE = "http://localhost:3000"
API_KEY = "sk_demo_00000000000000000000000000000000" # key demo, se re-siembr en cada
arranque
TIMEOUT = 30

# Una Session reutiliza la conexion TCP y aplica cabeceras comunes a cada request.
sesion = requests.Session()
sesion.headers.update({"x-api-key": API_KEY, "User-Agent": "MineriaWeb-2026-2/1.0"})
```

El "hola mundo": `GET /api/health` (sin credenciales)

`raise_for_status()` convierte un 4xx/5xx en excepción; `resp.json()` decodifica el cuerpo JSON a un `dict` de Python.

```

resp = requests.get(f"{BASE}/api/health", timeout=TIMEOUT) # sin header: este endpoint es abierto
resp.raise_for_status()

print("Status:", resp.status_code)
print("Content-Type:", resp.headers["Content-Type"])
print("Cuerpo:", resp.json())

```

```

Status: 200
Content-Type: application/json
Cuerpo: {'status': 'ok', 'service': 'tienda-virtual-api', 'time': '2026-10-07T01:42:19.824Z'}

```

Un solo recurso: GET /api/productos/1

Este endpoint sí exige el header `x-api-key` (ya cargado en la `Session`). El detalle de un producto trae, además de sus campos, sus `especificaciones`, sus `resenas` y el rating agregado.

```

resp = sesion.get(f"{BASE}/api/productos/1", timeout=TIMEOUT)
resp.raise_for_status()
producto = resp.json()

print("Claves del recurso:", list(producto.keys()))
print()
for campo in ("id", "codigo", "nombre", "categoria", "subcategoria", "precio", "stock"):
    print(f" {campo:12s} = {producto[campo]}")
print(f" especificaciones = {len(producto['especificaciones'])} filas")
print(f" resenas           = {len(producto['resenas'])}")
print(f" aggregateRating   = {producto['aggregateRating']}")

```

```

Claves del recurso: ['id', 'codigo', 'nombre', 'descripcion', 'categoria', 'subcategoria', 'precio', 'stock', 'especificaciones', 'aggregateRating', 'resenas']

```

```

id           = 1
codigo       = PROD-001
nombre       = Apple iPhone 15 Pro Max 256GB
categoria    = Smartphones
subcategoria = Gama Alta
precio       = 5599
stock        = 18
especificaciones = 5 filas
resenas      = 4
aggregateRating = {'ratingValue': 3.5, 'reviewCount': 4}

```

Una colección: GET /api/productos

Las APIs REST no devuelven "toda la tabla" de golpe: entregan **una página** y unos metadatos (`pageInfo`) para pedir el resto. Aquí pedimos solo la primera página de 5.

```

resp = sesion.get(f"{BASE}/api/productos", params={"page": 1, "pageSize": 5},
timeout=TIMEOUT)
resp.raise_for_status()
data = resp.json()

```

```

print("pageInfo:", data["pageInfo"])
print(f"La respuesta trae {len(data['items'])} de {data['pageInfo']['total']} productos\n")
for p in data["items"]:
    ar = p["aggregateRating"]
    print(f"    [{p['id']:>2}] {p['nombre'][:38]:38s} {p['categoria']:22s} S/ {p['precio']}")

```

pageInfo: {'page': 1, 'pageSize': 5, 'total': 90, 'totalPages': 18}

La respuesta trae 5 de 90 productos

[1]	Apple iPhone 15 Pro Max 256GB	Smartphones	S/ 5599
[2]	Apple iPhone 15 128GB	Smartphones	S/ 3799
[3]	Samsung Galaxy S24 Ultra 512GB	Smartphones	S/ 5299
[4]	Samsung Galaxy A55 128GB	Smartphones	S/ 1399
[5]	Xiaomi Redmi Note 13 Pro 256GB	Smartphones	S/ 999

Guardar la muestra en data/rest_muestra_productos.csv

```

DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT = os.path.join(DATA_DIR, "rest_muestra_productos.csv")

```

```

filas = []
for p in data["items"]:
    ar = p["aggregateRating"] or {}
    filas.append({
        "id": p["id"],
        "codigo": p["codigo"],
        "nombre": p["nombre"],
        "categoria": p["categoria"],
        "precio": p["precio"],
        "stock": p["stock"],
        "rating_promedio": ar.get("ratingValue", ""),
        "total_resenas": ar.get("reviewCount", ""),
    })

with open(OUTPUT, "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=filas[0].keys())
    writer.writeheader()
    writer.writerows(filas)

print(f"Guardadas {len(filas)} filas en {OUTPUT}")

```

Guardadas 5 filas en /private/tmp/claude-501/-Users-ferris-pucp-mineria-mineria-web-2026-2/1b003788-da84-40ac-86e0-5b0e2134c0b1/scratchpad/build/repo/sesion-de-clase-03/notebooks/./data/rest_muestra_productos.csv

Aquí pedimos una sola página a mano. En el siguiente notebook recorremos la colección **completa** automatizando la paginación.